

Permutation-Schedule Dominance for Two-Stage Assembly Flow Shops with Divisible Work

Annotated Edition for Executives and Practitioners

Preprint of the Journal of Scheduling submission (Paper A) — under review; not yet peer reviewed

Eric Bowman

King, Berlin, Germany

eric.bowman@king.com

ORCID: 0009-0003-9171-7217

How to read this document: the left column is the journal submission, word for word. The boxes in the right column are plain-language commentary for readers who are not scheduling theorists. The submission itself contains none of the boxes.

Abstract

Permutation schedules are known to dominate in the classical two-stage assembly flow shop for makespan and weighted completion time. We extend this dominance property to a divisible-work model in which each stage-1 machine contains k identical parallel workers, job workloads may differ by machine, and work is allocated integrally across workers. For any feasible schedule, we construct a permutation schedule whose stage-2 completion time is no larger for every job. This point-wise domination implies optimality of permutation schedules for every regular stage-2 objective, including makespan, total weighted completion time, and maximum lateness. The proof is constructive and yields the synchronized order explicitly from the release times of the original schedule. The construction is based on a single capacity inequality— k workers can complete at most kt units of work by time t —combined with a water-filling allocation that balances worker loads. The classical model is recovered when $k = 1$. Small instances are exhaustively verified computationally, and the core dominance and feasibility arguments are formalized in Lean 4.

Keywords: assembly flow shop, permutation schedule, divisible work, weighted completion time, flow shop scheduling

Mathematics Subject Classification: 90B35

1 Introduction

The two-stage assembly flow shop, introduced by Lee et al. (1993), consists of m parallel machines at stage 1, each processing all n jobs without preemption, followed by a single assembly machine at stage 2 that can begin job j only after all stage-1 machines have completed j .

Potts et al. (1995) proved the problem is NP-hard in the strong sense for $m \geq 2$ and that *permutation schedules dominate*: the search for a $C_{\max}$ -optimal schedule may be restricted to schedules where all stage-1 machines use the same job ordering, reducing the search space from $(n!)^m$ orderings to $n!$. Tozkan et al. (2003) extended permutation-schedule dominance to the total weighted flowtime objective. Hariri and Potts (1997) developed a branch-and-bound algorithm with dominance theorems for instances up to $n = 8000$, $m = 10$. Hadda et al. (2024)

What this paper proves, in one sentence: when several teams must each finish their part of every initiative before integration can complete it, there is always a schedule in which *all teams work in the same order* that delivers every single initiative at least as soon as whatever schedule you started with.

Why that matters: it ends the debate between “each team keeps its own priority order” and “one shared priority list.” Letting teams order work differently can never buy you anything — not on the final ship date, not on value-weighted delivery, not on the worst-case delay. This is a theorem with a machine-checked proof, not a simulation result or a rule of thumb.

“Assembly flow shop” is the scheduling-theory name for the fork-join pattern common in product development: multiple teams build in parallel, and nothing ships until every team has finished its part.

Translation: “machines” are teams; “jobs” are initiatives or features. Stage 1 is the parallel build phase — every team has its own piece of every initiative. Stage 2 is integration and release: it cannot start work on an initiative until every team has delivered its piece.

Two classical facts. First, finding the *best* order is computationally hard (“NP-hard”) — there is no shortcut algorithm. Second, and this is the useful one: whatever the best schedule is, a schedule where *every team uses the same order* is just as good. So you never need to consider mismatched team orders at all. With n initiatives and m teams there are $(n!)^m$ ways to give each team its own order, but only $n!$ shared orders — a staggering reduction in what you have to search.

contributed new dominance rules and an exact branch-and-bound, solving instances up to $n = 1000$, $m = 8$.

Koulamas and Kyparisis (2007) studied a variant with *uniform* parallel machines at stage 1 (machines with different speeds). Their model differs from ours: uniform machines have one processor per machine at varying speeds, whereas our model has k identical workers per machine sharing divisible work.

We distinguish several adjacent lines of work. The malleable-job literature (Drozdowski, 2009) studies divisible work on parallel processors but without the assembly constraint. Al-Anzi and Allahverdi (2007) study the two-stage assembly problem with setup times, using permutation dominance to restrict the heuristic search space. Hatami et al. (2013) consider distributed assembly permutation flow shops (multiple factories, not multiple workers per machine). Zhang and Chen (2022) study assembly scheduling with parallel teams and heterogeneous worker skills, in a multi-objective setting without dominance results. Komaki et al. (2019) survey assembly flow shop models; the divisible-work model studied here does not appear in their taxonomy. None of these states the dominance result proved here.

1.1 Contribution

We prove that permutation-schedule dominance extends to a divisible-work generalization where each stage-1 machine has $k \geq 1$ identical parallel workers and work requirements $w_{i,j}$ may differ across machines. Setting $k = 1$ recovers the classical model. The specific contributions are:

1. The extension from $k = 1$ to $k \geq 1$ workers per machine with integer work division, including the case $d_{i,j,\ell} \geq 0$ (some workers idle) with $w_{i,j} \geq 1$.
2. Machine-dependent work requirements $w_{i,j}$.
3. A shorter and more direct proof than the exchange-argument technique of Potts et al. (1995). Their proof transforms a schedule by pairwise job exchanges, tracking the assembly barrier at each step; ours constructs the permutation schedule directly from a single inequality without intermediate exchanges. The construction runs in $O(n \log n + mnk)$ time.

2 Model

Definition 1 (Divisible-Work Two-Stage Assembly Flow Shop). *An instance consists of:*

- n jobs, indexed $j \in [n] = \{1, \dots, n\}$.
- m stage-1 machines, each with k identical workers ($k \geq 1$).
- One stage-2 machine (with any number of workers; the stage-2 schedule is inherited unchanged, so the stage-2 structure does not affect the dominance result).
- Work requirements $w_{i,j} \in \mathbb{Z}_{>0}$ for each machine $i \in [m]$ and job $j \in [n]$.
- Stage-2 work requirements $q_j \in \mathbb{Z}_{>0}$ for each job j .

Definition 2 (Schedule). *A schedule $S = (\sigma, \mathbf{d}, \tau, \mathbf{e})$ consists of:*

This has been known since 1995 — but only for teams that are *single machines*: one server, one piece of work at a time, fixed durations.

What is genuinely new here. Real teams are not single machines: a team of k people can split an initiative's work among its members. This paper proves the shared-order result for that model — teams of k interchangeable workers dividing whole units of work — which turns out not to appear anywhere in the scheduling literature. The paragraph on the left walks the neighbouring research areas and explains why each one is a different problem. (Fifteen-plus targeted literature searches sit behind the final sentence.)

The three contributions, translated. (1) Teams instead of single machines: each team has k members who split each initiative's work between them, in whole units; it is fine for some members to sit out an initiative. (2) Each team can have a *different* workload for the same initiative — backend might have 6 units of work where art has 2. (3) The proof is constructive and cheap: given any schedule, you can *compute* the shared-order schedule that beats it, in roughly “sort the initiatives, then deal out the work” time. The 1995 proof transformed a schedule swap by swap; this one builds the answer in a single step.

The cast: n initiatives, m build teams of k people each, one integration stage. $w_{i,j}$ is team i 's workload for initiative j (think person-days), q_j is the integration workload. Workloads are whole numbers — work is handed out in indivisible units.

- Permutations $\sigma_i \in \mathfrak{S}_n$ for each machine i : machine i processes jobs in order $\sigma_i(1), \dots, \sigma_i(n)$.
- Work divisions $d_{i,j,\ell} \in \mathbb{Z}_{\geq 0}$ for each machine i , job j , worker $\ell \in [k]$, satisfying $\sum_{\ell} d_{i,j,\ell} = w_{i,j}$.
- A permutation $\tau \in \mathfrak{S}_n$ and work divisions $e_{j,\ell}$ for stage 2, with $\sum_{\ell} e_{j,\ell} = q_j$.

All processing is non-preemptive: each worker completes its assigned portion of a job before starting the next.

Definition 3 (Completion Times). Worker ℓ on machine i processes jobs sequentially in order σ_i . The cumulative finish time after position s is $F_{i,\ell,s} = \sum_{t=1}^s d_{i,\sigma_i(t),\ell}$. The completion time of job j on machine i is $C_{i,j} = \max_{\ell} F_{i,\ell,\sigma_i^{-1}(j)}$. All completion times are non-negative integers.

The release time of job j to stage 2 is $r_j = \max_{i \in [m]} C_{i,j}$.

Stage-2 worker ℓ starts job $\tau(s)$ at time $\max(r_{\tau(s)}, F_{\ell,s-1}^2)$ and finishes at $F_{\ell,s}^2 = \max(r_{\tau(s)}, F_{\ell,s-1}^2) + e_{\tau(s),\ell}$, with $F_{\ell,0}^2 = 0$. We write $C_j^{(2)} = \max_{\ell} F_{\ell,\tau^{-1}(j)}^2$ for the stage-2 completion time of job j , and $C_{\max}(S) = \max_j C_j^{(2)}$ for the makespan.

Definition 4 (Permutation Schedule). A schedule is a permutation schedule if $\sigma_1 = \sigma_2 = \dots = \sigma_m$.

In the three-field notation of Graham et al. (1979), the problem is $\text{FH2}(P_{m \times k}, P_{k'}) \mid \text{assembly, div} \mid C_{\max}$, where “div” denotes divisible integer work.

3 Main Result

Theorem 5 (Pointwise Stage-2 Domination). For every schedule S , there exists a permutation schedule S^* with $C_j^{(2)}(S^*) \leq C_j^{(2)}(S)$ for every job j .

Corollary 6. Permutation schedules are optimal for every regular (non-decreasing) function of stage-2 completion times, including $C_{\max}$, $\sum_j v_j C_j^{(2)}$ for any $v_j > 0$, and $\max_j (C_j^{(2)} - d_j)$ for any due dates d_j .

The proof of Theorem 5 uses two lemmas.

Lemma 7 (Capacity Bound). Let S be any schedule with release times r_j . Let π be a permutation sorting jobs by non-decreasing release time: $r_{\pi(1)} \leq \dots \leq r_{\pi(n)}$. Then for every machine i and every $s \in [n]$:

$$W_i(s) := \sum_{t=1}^s w_{i,\pi(t)} \leq k \cdot r_{\pi(s)}.$$

Proof. In schedule S , machine i completes job $\pi(t)$ by time $C_{i,\pi(t)} \leq r_{\pi(t)} \leq r_{\pi(s)}$ for each $t \leq s$. Hence by time $r_{\pi(s)}$, machine i has completed at least jobs $\{\pi(1), \dots, \pi(s)\}$, whose total work is $W_i(s)$. Since k workers each contribute at most $r_{\pi(s)}$ time units: $W_i(s) \leq k \cdot r_{\pi(s)}$. $\square$

Lemma 8 (Water-Filling Feasibility). Given machine i processing jobs in order π , allocating work one unit at a time to the currently least-loaded worker achieves completion time $\lceil W_i(s)/k \rceil$ at each position s .

Proof. By induction on s . The invariant: after position s , worker loads

A schedule is four decisions: the order each team works in (σ_i — note each team may have its own order), how each team splits each initiative among its members (d), and the same two decisions for integration (τ, e). “Non-preemptive” means nobody parks a half-done assignment to pick up another one.

Three clocks. Each worker’s clock just adds up their assignments in order. An initiative is done on a team when the last assigned member finishes ($C_{i,j}$). It is released to integration when the last team finishes it (r_j) — the fork-join barrier. Integration then handles initiatives in its own order: each piece starts as soon as the initiative is released and the integration worker is free, whichever is later. That “later-of-the-two” recurrence is the only fact about stage 2 the whole proof will need.

Permutation schedule = one shared priority list. Every team works the initiatives in the same order. The three-field string is the scheduling community’s catalogue number for the problem — useful for literature search, nothing more.

The headline result. Take any schedule — mismatched team orders, lopsided work splits, anything. There is a shared-list schedule under which every individual initiative finishes integration at least as soon. Not just the last one: every one. “Pointwise” is the strong part, and it is what makes the corollary below free.

One theorem, every sensible KPI. Any metric that never gets worse when initiatives finish sooner — final ship date, value-weighted delivery, worst slip against due dates — is optimized by some shared-list schedule. Because every initiative’s date improves (or holds), you do not need a separate argument per metric.

The engine of the whole proof. List the initiatives in the order the original schedule released them. By the moment the s -th one was released, every team had necessarily finished the first s — and a team of k people working for t time units can produce at most $k \cdot t$ units of work. So the workload of those first s initiatives fits under the line $k \cdot r_{\pi(s)}$, on every team. That is the only thing the original schedule is ever asked to provide: its own release times become budget ceilings. The proof is pure counting — no scheduling cleverness anywhere.

Keep every plate level. “Water-filling” is the dealing rule: hand out work one unit at a time, always to whoever currently has the least. Then no two team members’ workloads ever differ by more than one unit, so the team finishes its first s

lie in $\{\lceil W_i(s)/k \rceil, \lfloor W_i(s)/k \rfloor\}$ (loads differ by at most 1 and sum to $W_i(s)$).

At position s , distribute $w_{i,\pi(s)}$ units one at a time, each to a currently least-loaded worker. Since the invariant holds at $s-1$, loads before allocation differ by at most 1. Adding units one-at-a-time to the minimum maintains the spread ≤ 1 . The total becomes $W_i(s)$, so the maximum is $\lceil W_i(s)/k \rceil$.

When $w_{i,\pi(s)} < k$, some workers are idle for this job ($d_{i,\pi(s),\ell} = 0$), which the model permits. When $w_{i,\pi(s)} \geq k$, the allocation can also be arranged so that every worker receives at least one unit: give each worker one unit first (adding one unit to every load preserves the spread), then distribute the remaining $w_{i,\pi(s)} - k$ units least-loaded-first.

The division is also available in closed form: after position s , exactly $W_i(s) \bmod k$ workers hold $\lceil W_i(s)/k \rceil$ units and the rest hold $\lfloor W_i(s)/k \rfloor$, so the divisions for each job are computable in $O(k)$ time. This gives the $O(n \log n + mnk)$ construction time claimed in Section 1. $\square$

Proof of Theorem 5. Given schedule S with release times r_j , let π sort jobs by non-decreasing r_j . Construct S^* :

1. Every stage-1 machine uses permutation π .
2. Each machine i independently uses water-filling (Lemma 8) to allocate $w_{i,\pi(s)}$.
3. Stage 2 uses the same τ and $\mathbf{e}$ as S .

By Lemma 8, $C_{i,\pi(s)}^* = \lceil W_i(s)/k \rceil$. By Lemma 7, $W_i(s) \leq k \cdot r_{\pi(s)}$. Since $r_{\pi(s)}$ is a non-negative integer: $\lceil W_i(s)/k \rceil \leq r_{\pi(s)}$ for all i, s . Therefore $r_j^* \leq r_j$ for all j .

Since $r_j^* \leq r_j$ and the stage-2 schedule is unchanged, by induction on stage-2 positions: $F_{\ell,s}^{2*} \leq F_{\ell,s}^2$ for all ℓ, s . Therefore $C_j^{(2)}(S^*) \leq C_j^{(2)}(S)$ for every j . $\square$

Remark 9 (Classical recovery). *When $k = 1$, water-filling is the identity and the theorem recovers the permutation-schedule dominance of Potts et al. (1995).*

Remark 10 (The permutation π). *The construction uses π sorted by the release times of the given schedule S . The result is existential: for every S , there exists a permutation schedule at least as good.*

Remark 11 (Stage-2 re-optimization). *The construction inherits stage 2 unchanged. Re-optimizing the stage-2 work divisions or ordering can only improve the result but is not needed for dominance.*

Remark 12 (Scope of the construction). *Four generalizations follow from the same proof verbatim. (i) Machine-dependent worker counts: if machine i has k_i workers, Lemma 7 gives $W_i(s) \leq k_i r_{\pi(s)}$ and water-filling yields $\lceil W_i(s)/k_i \rceil \leq r_{\pi(s)}$; nothing else changes. (ii) Real-valued work: with divisible real-valued workloads the integrality step is unnecessary—equal splitting gives completion $W_i(s)/k \leq r_{\pi(s)}$ directly. (iii) Inserted idle time: if schedules may idle workers, Lemma 7 still holds (each worker completes at most $r_{\pi(s)}$ units of work by time $r_{\pi(s)}$), and the constructed S^* uses no idle time, so the domination is unchanged. (iv) General downstream structure: the proof of Theorem 5 uses only that stage-2 completion times are componentwise non-decreasing in the release-time vector, so the single assembly stage may be replaced by any downstream system with this monotonicity property.*

initiatives at exactly $\lceil W_i(s)/k \rceil$ — total work divided by team size, rounded up to a whole unit. That is the soonest any k -person team could possibly finish that much work, so the rule wastes nothing.

The closing paragraph notes you never need to deal unit by unit in practice: the final split has a simple closed form, which is why constructing the whole schedule is cheap.

The whole proof is a three-step chain. (1) The old schedule's release times are budget ceilings: the work it released by each date fits under $k \cdot t$ on every team (capacity bound). (2) The new schedule — every team works the shared order π , dealing work plate-leveiling style — finishes each prefix at total-work-over- k , rounded up; and because the ceilings are whole numbers, the round-up still fits under them. So every initiative is released *no later than before*. (3) Integration is untouched, and its “start when released and free” recurrence can only give earlier finishes when its inputs arrive earlier. Done.

The quiet trick is in step 2: $W \leq k r$ only gives $W/k \leq r$, and rounding up could break that — except r is a whole number of time units, so $\lceil W/k \rceil \leq r$ holds anyway. That single line is where the integer model earns its keep.

Sanity check and a caveat. With one-person teams the construction collapses to the known 1995 theorem — the new result strictly contains the old one. The caveat: the winning shared order is computed *from* the schedule it beats; there is no single magic order that beats everything. For decision-making that does not matter: it means the search for the best schedule, under any regular metric, can be restricted to shared-list schedules.

The theorem travels further than its statement. Teams of different sizes: fine. Work measured in fractional units instead of whole ones: fine (the proof gets easier). Schedules where people sometimes sit idle: fine. And — the one with the most practical reach — integration can be replaced by any downstream process, of any shape, as long as it has the property that inputs arriving earlier never make its outputs later. Whatever your release process looks like, if earlier hand-offs never hurt it, the shared list wins.

Example. Let $n = 3, m = 2, k = 2$. Machine 1: $(w_{1,A}, w_{1,B}, w_{1,C}) = (6, 4, 2)$; machine 2: $(w_{2,A}, w_{2,B}, w_{2,C}) = (2, 6, 4)$.

Schedule S : machine 1 uses A, B, C ; machine 2 uses B, C, A . Water-filling gives: machine 1 completes A at 3, B at 5, C at 6; machine 2 completes B at 3, C at 5, A at 6. Release times: $r_A = 6, r_B = 5, r_C = 6$.

Sort: $\pi = (B, A, C)$. Capacity bound holds: $W_1(1) = 4 \leq 10, W_2(1) = 6 \leq 10$, etc. Water-filling with π : $r_B^* = 3 \leq 5, r_A^* = 5 \leq 6, r_C^* = 6 \leq 6$. Pointwise domination.

The example in words. Two teams of two, three initiatives, mismatched orders. Under the original schedule the initiatives reach integration at times 6, 5, 6. Sort by those dates (B , then A , then C), put both teams on that shared order, deal the work evenly — and the release dates become 3, 5, 6. Initiative B arrives two units sooner, A one unit sooner, C no later. Nothing got worse; two of three got better. That is the theorem in miniature.

4 Computational Verification and Lean 4 Formalization

We verified Theorem 5 by exhaustive enumeration across 27 homogeneous-work configurations ($w_{i,j} = w_j$; 3,846,136 schedules) and 12 heterogeneous-work configurations ($w_{i,j}$ varying across machines; 167,120 schedules), with $n \in \{2, \dots, 5\}, m \in \{2, 3\}, k \in \{2, 3\}$. In every case, a permutation schedule achieved the pointwise minimum.

Each step of the proof is formalized in Lean 4 (v4.29.0, Mathlib) with zero unresolved axioms: the capacity bound (Lemma 7) is derived from a concrete machine model with per-worker cumulative finish times; the water-filling allocation (Lemma 8) is constructed explicitly—workers take work units round-robin—and shown to achieve $\lceil W_i(s)/k \rceil$ at every position; and the release-time domination composes with stage-2 monotonicity to give the pointwise conclusion.¹ The development contains no unproven assumptions (no **sorry**). It also includes a machine-checked counterexample showing that the bounds available to a pairwise-interchange argument cannot establish the theorem at the release-time abstraction, which motivated the direct construction.

Belt. For 39 small scenario families — over four million schedules in total — a program enumerated *every possible schedule* (not samples) and confirmed the theorem’s prediction each time, with zero exceptions. This cannot prove the theorem (no finite check can), but it would have caught an error in it.

5 Conclusion

For any schedule in the divisible-work two-stage assembly flow shop, we have constructed a permutation schedule with $C_j^{(2)}(S^*) \leq C_j^{(2)}(S)$ for every job j , using a capacity bound and water-filling. This extends the classical results of Potts et al. (1995) and Tozkapan et al. (2003) from $k = 1$ to arbitrary k , with machine-dependent work requirements.

Open questions include whether the result extends to deeper assembly structures (more than two stages), stochastic processing times, or sequence-dependent setup costs.

And braces. Lean is a proof assistant: a program that checks every logical step of a proof down to the axioms of mathematics, the way a compiler checks types. Every lemma in the paper has been verified this way, end to end. So the proof’s correctness does not rest on the author’s algebra, or on a referee’s patience — a machine has checked all of it. (“No sorry” is Lean jargon: a sorry is an IOU where a proof step is assumed rather than proved. There are none.)

Declarations

Funding. This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Competing interests. The author declares no competing interests.

Author contributions. The sole author conceived the work, developed the proofs and formalization, and wrote the manuscript.

Data availability. The verification data supporting the quantitative claims of Section 4 are available in the repository below.

Code availability. The Lean 4 formalization and Scala verification code are available at <https://github.com/ebowman/synchronization-paper-artifact> and archived at <https://doi.org/10.5281/zenodo.20075388> (this submission corresponds to version 1.1.0).

Where the boundary lies. Two stages — parallel build, then integration — is exactly where the clean result lives. For pipelines of three or more stages the pointwise guarantee provably *fails* (computer search finds counterexamples), and mapping precisely where dominance survives and where it breaks is the subject of the companion paper (Paper B), also submitted to the Journal of Scheduling.

Everything is reproducible. The proof checker, the four-million-schedule enumeration, and a registry tracing every quantitative claim in the paper to the code that verifies it are public and permanently archived with a DOI.

¹Source code, verification data, and Lean formalization: <https://github.com/ebowman/synchronization-paper-artifact>.

Generative AI disclosure. During the preparation of this work, the author used Claude for critique of exposition, structural feedback, and language revision. The author reviewed and edited the manuscript and takes full responsibility for the content.

References

- C.-Y. Lee, T. C. E. Cheng, and B. M. T. Lin. Minimizing the makespan in the 3-machine assembly-type flowshop scheduling problem. *Management Science*, 39(5):616–625, 1993.
- C. N. Potts, S. V. Sevast’janov, V. A. Strusevich, L. N. Van Wassenhove, and C. M. Zwaneveld. The two-stage assembly scheduling problem: Complexity and approximation. *Operations Research*, 43(2):346–355, 1995.
- A. Tozkapan, Ö. Kırça, and C.-S. Chung. A branch and bound algorithm to minimize the total weighted flowtime for the two-stage assembly scheduling problem. *Computers & Operations Research*, 30(2):309–320, 2003.
- A. M. A. Hariri and C. N. Potts. A branch and bound algorithm for the two-stage assembly scheduling problem. *European Journal of Operational Research*, 103(3):547–556, 1997.
- H. Hadda, N. Dridi, and S. Hajri-Gabouj. On the two-stage assembly flow shop problem. *TOP*, 32(2):224–244, 2024.
- C. Koulamas and G. J. Kyparisis. A note on the two-stage assembly flow shop scheduling problem with uniform parallel machines. *European Journal of Operational Research*, 182(2):945–951, 2007.
- M. Drozdowski. *Scheduling for Parallel Processing*. Springer, London, 2009.
- F. S. Al-Anzi and A. Allahverdi. A self-adaptive differential evolution heuristic for two-stage assembly scheduling problem to minimize maximum lateness with setup times. *European Journal of Operational Research*, 182(1):80–94, 2007.
- S. Hatami, R. Ruiz, and C. Andrés-Romano. The distributed assembly permutation flowshop scheduling problem. *International Journal of Production Research*, 51(17):5292–5308, 2013.
- Z. Zhang and R. Chen. Multi-objective complex product assembly scheduling problem considering parallel team and worker skills. *Journal of Manufacturing Systems*, 62:876–893, 2022.
- G. M. Komaki, S. Sheikh, and B. Malakooti. Flow shop scheduling problems with assembly operations: A review and new trends. *International Journal of Production Research*, 57(10):2926–2955, 2019.
- R. L. Graham, E. L. Lawler, J. K. Lenstra, and A. H. G. Rinnooy Kan. Optimization and approximation in deterministic sequencing and scheduling: A survey. *Annals of Discrete Mathematics*, 5:287–326, 1979.